ジオメトリック・インテリジェンス

ケーススタディ補助文書 03

実装スクリプト集

― 光稜精機株式会社 4 年間の戦場に投入された Python 実装 ―

本書は、ケーススタディ本編「光稜の不動点」に登場する Python コード断片を、
GI 理論のパイプラインの流れに沿って整理し、
各コード片の数学的背景、運用上の意図、実装上の注意を解説した実装ガイドである。

Étale Cohomology

2026 年 10 月

License: CC BY 4.0

本書の位置づけ

本書はケーススタディ「光稜の不動点」の補助文書 03 である。関連文書として以下がある。

- 本編：「光稜の不動点 — 光稜精機株式会社 4 年間の戦場」
- 補助文書 01：本編と GI 理論の対応関係
- 補助文書 02：本編で使用された数学の解説
- 本書（補助文書 03）：実装スクリプト集
- 補助文書 04：業界知識

各節の「参照」では、GI 理論のコンパニオン（Vol.1 / Vol.2）の該当章と、補助文書 02 で番号を付けた参考記事（Qiita）を示す。コンパニオンは次の URL で公開されている。

- Vol.1：https://github.com/EtaleCohomology/geometric-intelligence-saga/blob/main/companion/GI_Companion_Vol1_JP.pdf
- Vol.2：https://github.com/EtaleCohomology/geometric-intelligence-saga/blob/main/companion/GI_Companion_Vol2_JP.pdf

掲載コードについて

本書に掲載したコード断片は、本編に登場したコードを忠実に再録したものである。本編のコードは、物語の中で数式とコードを橋渡しすることを優先しているため、そのままでは実行できない箇所（外部関数の呼び出し、省略された数学的処理、本編の数値を出力として示した部分など）を含む。

また、教育目的の簡略化のために、数学的に厳密でない箇所もある。本書ではそれらを書き換えず、各コード片の「実装上の注意」で指摘し、正しい実装の考え方を補足した。補足の参照実装は付録 B にまとめている。付録 B のうち B.5〜B.8 は、実際に実行して出力を確認したものである。本編の物語の数値は、本書の注意を反映した実装による再計算の結果ではない点に留意されたい。なお、等幅フォントで表示できない一部の記号（×印、下付き・上付きの文字など）は、意味を変えずに置き換えた。

ライセンスとクレジット

© 2026 Étale Cohomology. 本書は Creative Commons Attribution 4.0 International（CC BY 4.0）のもとで公開される。

目次

1　はじめに　4

1.1　パイプライン全体像 4
1.2　本作のコードの特徴 4
1.3　各コード片の構成 4
1.4　推奨ライブラリ 4

2　データ基盤と 2 層アーキテクチャ　5

2.1　光稜精機の 2 層アーキテクチャ 5
2.2　データ基盤の全体像 6

3　ステップ 1〜2：多様体の構築と命題 1.1　8

3.1　命題 1.1 の検証 — 第 1 回試行（不合格） 8
3.2　シリコンサイクルの位相角の推定 9
3.3　命題 1.1 の検証 — 第 2 回試行 10
3.4　レシピから物理量への変換 11
3.5　命題 1.1 の検証 — 第 3 回試行（合格） 12

4　ステップ 3〜4：引き戻し計量とスカラー曲率　13

4.1　引き戻し計量と条件数 13
4.2　スカラー曲率マップ 13

5　AI エージェントと身体化エージェント　15

5.1　14 体の AI エージェント・ロスター 15
5.2　身体化エージェントのアラート（Agent_01） 16
5.3　非身体化エージェントの評価（Agent_08） 17
5.4　知覚空間の位相 18

6　パーシステント・ホモロジー　21

6.1　9 つの危機のパーシステント・ホモロジー 21

7　多人数ゲーム理論　22

7.1　4 者のナッシュ均衡 22

8　手術理論によるポートフォリオの再編　23

8.1　手術計画 23

9　シンプレクティック積分　24

9.1　8 年間の航海図 24

10 層理論による内部の不整合の検知 **25**

10.1 組織の層ラプラシアン 25

11 座標系の変換（本編の呼称：ゲージ変換） **26**

11.1 秘密の回転による流出データの無効化 26

12 形式検証と ZKML **28**

12.1 形式検証の結果 28

12.2 ZKML 証明の生成 28

付録 A　推奨ライブラリ一覧 **30**

付録 B　補足の参照実装 **31**

付録 C　コード索引 **36**

第1章　はじめに

1.1　パイプライン全体像

GI 理論のパイプラインは、データ収集（ステップ 1）、VAE による多様体構築（2）、引き戻し計量（3）、曲率（4）、リー微分・リー括弧（5）、MC ドロップアウト（6）、SHAP と DML（7）、Neural ODE と最適制御（8）、品質チェック（9）、レポート生成（10）の 10 ステップから成る。本書の章立てはこの順序に従い、その後に AI エージェント、幾何学的拡張、形式検証と ZKML を置いた。

1.2　本作のコードの特徴

1 作目・2 作目のコードと比べ、本作には次の特徴がある。

- **周期構造と座標の選び方**。シリコンサイクルの瞬時位相をヒルベルト変換で推定する関数（第 3.2 節）と、装置のレシピをプラズマの物理量に変換する関数（第 3.4 節）。
- **2 層アーキテクチャと身体化エージェント**。月次の経営環境多様体と、秒単位の装置デジタルツインをつなぐ構成（第 2 章）と、物理法則に基づく身体化エージェントのアラート（第 5 章）。
- **組織の整合性の検証と防御**。層ラプラシアンによる社内の不整合の検知（第 10 章）と、座標系の秘密の回転による流出データの無効化（第 11 章）。

1.3　各コード片の構成

各節は「背景」「コード」「解説」「実装上の注意」「参照」の順に構成する。「背景」は本編のどの場面でそのコードが登場したか、「解説」はコードが何を計算しているか、「実装上の注意」は実運用で修正・補強すべき点を述べる。

1.4　推奨ライブラリ

本編のコードは、NumPy、SciPy、PyTorch、torchdiffeq、scikit-learn、EconML、SHAP、ripser / persim、Gambit、z3-solver を前提としている。詳細は付録 A を参照。

第2章　データ基盤と 2 層アーキテクチャ

2.1　光稜精機の 2 層アーキテクチャ

出典：本編第 1 章

背景　第 1 章「リームとの最初の接触」。半導体製造装置メーカーのデータが、月次の経営データと秒単位の装置データという、時間の尺度がまったく異なる 2 種類から成ることを受けて、リームが 2 層の構成を示す。

コード

```
光稜精機の2層アーキテクチャ
──────────────────────────────────────────────────────
【上層】経営環境多様体（月次更新）
  次元：約130指標 → VAEで13〜16次元の潜在空間に圧縮
  データ：財務、受注、サプライチェーン、地政学、規制、
          顧客動向、競合動向、サイバー脅威
  搭載エージェント：非身体化エージェント
  ↕ 接続：リー微分による影響伝播
【下層】装置デジタルツイン群（リアルタイム更新）
  次元：装置1台あたり数千パラメータ × 数百台
  データ：プラズマ状態、ガス流量、チャンバー圧力、
          ウェハー膜厚、エッチング深さ、RF電力
  搭載エージェント：身体化エージェント
──────────────────────────────────────────────────────
身体化エージェントが装置の物理的異常を検知
  → 上層の多様体に異常信号として伝播
  → 非身体化エージェントが経営インパクトを分析
  → リー括弧で「装置の異常 × 地政学の変動」の創発効果を計算
```

解説　上層は約 130 指標の月次データから作る経営環境多様体で、非身体化エージェントが動く。下層は装置ごとの数千パラメータを秒単位で扱うデジタルツイン群で、身体化エージェントが動く。下層で検知した物理的な異常を上層に伝え、経営への影響を分析し、「装置の異常」と「地政学の変動」の組み合わせの効果（創発効果）をリー括弧で評価する。

実装上の注意

二つの層は時間の尺度が 6 桁以上違うため、下層の生データをそのまま上層に入れることはできない。下層の異常を「月ごとの異常の件数や深刻度」「装置停止の時間」などの指標に集約してから上層に渡すか、イベントとして上層の多様体の上の点（またはベクトル場）に変換する設計が必要である。また、リー括弧で創発効果を測るには、二つのベクトル場がいずれも状態に依存していなければならない。定数のベクトル場どうしのリー括弧は常に 0 になる（付録 B.2）。

参照　コンパニオン Vol.2 第 6 章（デジタルツイン×AI Agent）、第 7 章（Agent 設計の体系）。参考記事 [12][13]。

2.2　データ基盤の全体像

出典：本編第 2 章

背景　第 2 章「半導体装置メーカーのデータ収集」。瀬川・長谷部・伊藤の 3 名とともに、多様体の構築に使うデータを整理する。

コード

```
光稜精機 データ基盤の全体像
═══════════════════════════════════════════════════════════

1. SAP S/4HANA + 光稜独自ERP（本社）
   内容：受注・売上・原価・在庫（装置別・顧客別・地域別）、
         部品調達、人件費、R&D投資
   期間：2013年～現在（12年分）
   品質：★★★★☆
   問題点：シリコンサイクルの好況・不況で受注額が2倍以上変動。
           2019年（不況底）と2021年（好況ピーク）のデータは
           同じ「月次売上」でも経営の意味がまったく異なる。

2. 装置プロセスデータ（テクノロジーセンター）
   内容：エッチング装置のプラズマ状態モニタリング
         （電子密度、イオンエネルギー、ガス流量8種、
         チャンバー圧力、RF電力2チャネル、温度12点）
   期間：2018年～現在（7年分）
   データ量：装置1台あたり1日約30GB
   品質：★★★★★（物理センサーの精度は極めて高い）
   問題点：装置の技術世代が変わるとパラメータ体系が変わる。
           2018年の14nm世代と2025年の3nm世代では
           計測項目すら異なる。

3. 顧客装置稼働データ（カスタマーサポート経由）
   内容：出荷済み装置の稼働率、エラーログ、部品交換履歴
   期間：2015年～現在（10年分）
   品質：★★★☆☆
   問題点：顧客ごとに完全にサイロ化。TSMC向けデータと
           Samsung向けデータは物理的に別サーバー。
           歩留まり（イールド）データは顧客の最高機密であり
           光稜には一切共有されない。

4. Bloomberg / Refinitiv（市場データ）
   内容：半導体関連株価指数（SOX指数）、シリコンウェハー価格、
         レアガス（ネオン、クリプトン）価格、為替
   期間：2010年～現在（15年分）
   品質：★★★★★
```

```
5. 地政学・規制データベース
   内容：米国BIS EAR改定履歴、Entity List更新、
         日本経産省の半導体戦略文書、EU Chips Act、
         中国の半導体内製化政策
   期間：2018年〜現在（7年分、赤羽CROの部門が管理）
   品質：★★★★☆（構造化されている）
   問題点：規制変更の「予兆」データが体系化されていない

6. SIEMログ（サイバーセキュリティ）
   内容：ITネットワーク＋OT（クリーンルーム制御系）ネットワーク
   期間：2021年〜現在（4年分）
   品質：★★★☆☆（ITは良好、OTは不十分）
──────────────────────────────────────────────────────────────
```

解説　財務・受注・サプライチェーン・規制・地政学・顧客・競合・サイバーなどの指標を月次でそろえ、装置データは集約したうえで加える構成である。

実装上の注意

顧客ごとのデータ（とくに歩留まりや装置の稼働データ）は、同じ社内でも厳格に分離されている（補助文書 04 第 3.2 節）。分析の基盤を作る段階で、どのデータを誰がどの形で使えるかを決め、必要に応じて暗号化したまま計算し、ZKML で計算の正しさを示す設計にする（第 12 章）。指標ごとに更新の頻度や欠損の扱いが異なるため、月次への集約の規則を文書化しておくこと。

参照　コンパニオン Vol.1 第 9 章（データ収集）。

第3章　ステップ 1〜2：多様体の構築と命題 1.1

3.1　命題 1.1 の検証 ― 第 1 回試行（不合格）

出典：本編第 2 章

背景　第 2 章「命題 1.1 ― シリコンサイクルの壁」。固有次元 14.2 を推定し、潜在次元 14 の VAE を訓練したが、3 条件のうち 2 つが不合格となった。

コード

```
# ===== 命題1.1検証結果 ―― 光稜精機 第1回試行 =====
d_hat = estimate_intrinsic_dimension(X_normalized_koryo)
print(f"推定固有次元数：{d_hat:.1f}")
# → 14.2
# VAE（潜在次元14）を3,000エポック、22時間で訓練
# 条件1（再構成忠実度）：
mse = 0.0082
print(f"条件1：MSE = {mse:.4f} < 0.01 → 合格 ✓")
# 条件2（潜在空間の滑らかさ）：
knn = 0.63
print(f"条件2：KNN保存率 = {knn:.2f} >= 0.90 → 不合格 ×")
# 条件3（ヤコビアンの非退化性）：
min_sv = 0.00003
print(f"条件3：最小特異値 = {min_sv:.5f} >= 0.001 → 不合格 ×")
推定固有次元数：14.2
条件1：MSE = 0.0082 < 0.01 → 合格 ✓
条件2：KNN保存率 = 0.63 >= 0.90 → 不合格 ×
条件3：最小特異値 = 0.00003 >= 0.001 → 不合格 ×
```

解説　命題 1.1 の 3 条件は、(1) 再構成誤差（MSE）が小さいこと、(2) 潜在空間でデータの近傍関係が保たれていること（KNN 保存率）、(3) デコーダのヤコビ行列が退化していないこと（最小特異値）である。第 1 回は、好況期・不況期・遷移期が別々の島に分かれて学習され、近傍関係が崩れ（0.63）、島の境目でヤコビ行列がほぼ退化した（0.00003）。

実装上の注意

掲載コードは、本編の数値を代入して判定結果を表示しているだけで、実際の計算は省略されている。固有次元の推定には TwoNN 法や最尤推定法などを使い、推定値の幅も確認する。KNN 保存率は「データ空間で近い k 個の点のうち、潜在空間でも近い k 個に入っている割合」として定義を明記すること。最小特異値は 1 点ではなく、多数のサンプル点での分布（とくに下側の分位点）で評価する。

参照　コンパニオン Vol.1 第 5 章（VAE と滑らかな潜在多様体の構成）。

3.2　シリコンサイクルの位相角の推定

出典：本編第 2 章

背景　第 2 章。瀬川凛太郎が、プラズマ物理で日常的に使うヒルベルト変換を、シリコンサイクルの「いまの位置」の推定に応用することを提案する。

コード

```
# ===== シリコンサイクルの位相角推定 ―― 瀬川凛太郎の提案 =====
from scipy.signal import hilbert
import numpy as np
def estimate_silicon_cycle_phase(order_data, period_range=(42, 54)):
    """
    シリコンサイクルの瞬時位相をヒルベルト変換で推定する。
    ◆ ヒルベルト変換とは：
      時系列データの「瞬時的な振動の位相」を推定する手法。
      周期が一定でない振動（シリコンサイクルのように
      3.5年〜4.5年で揺らぐ周期）にも適用できる。

    ◆ なぜこれが必要か：
      2019年の不況底と2023年の不況底は、カレンダー上では4年離れている。
      しかしシリコンサイクル上では「同じフェーズ」にいる。
      この「サイクル上のどこにいるか」を追加の指標として
      VAEに投入すれば、サイクルの異なるフェーズを
      "別世界"ではなく"同じ世界の異なる位置"として学習できる。

    瀬川凛太郎の知見：
      プラズマ物理学では、プラズマ振動の瞬時周波数を
      推定するためにヒルベルト変換を日常的に使用する。
      同じ数学がシリコンサイクルにも適用できる。
    """
    # バンドパスフィルタでシリコンサイクル帯域を抽出
    # period_range = (42, 54) ヶ月 = 3.5年〜4.5年
    from scipy.signal import butter, filtfilt
    fs = 12  # サンプリング周波数（月次データ = 年12回）
    low = 1.0 / (period_range[1] / 12)
    high = 1.0 / (period_range[0] / 12)
    b, a = butter(3, [low / (fs/2), high / (fs/2)], btype='band')
    cycle_component = filtfilt(b, a, order_data)
    # ヒルベルト変換で瞬時位相を推定
    analytic_signal = hilbert(cycle_component)
    instantaneous_phase = np.angle(analytic_signal)  # -π〜πの範囲
    # cos(位相)とsin(位相)の2指標として返す
    #  （位相角をそのまま使うと-πとπの間に不連続が生じるため）
    return np.cos(instantaneous_phase), np.sin(instantaneous_phase)
# 光稜の受注データからシリコンサイクルの位相を推定
cos_phase, sin_phase = estimate_silicon_cycle_phase(koryo_order_data)
print(f"シリコンサイクル位相推定完了")
```

```
print(f"  現在の位相角：{np.arctan2(sin_phase[-1], cos_phase[-1]) * 180 / np.pi:.1f}°")
print(f"  → サイクルの約270°の位置（不況底からの回復初期に相当）")
シリコンサイクル位相推定完了
  現在の位相角：268.3°
  → サイクルの約270°の位置（不況底からの回復初期に相当）
```

解説　受注データからシリコンサイクルの帯域（周期 42〜54 か月）だけをバンドパス・フィルタで取り出し、ヒルベルト変換で解析信号を作って瞬時位相を求める。位相そのものではなく $(\cos\phi, \sin\phi)$ の 2 指標として VAE に加えることで、359° と 1° が隣り合う円の構造を保つ（補助文書 02 第 4 章）。

実装上の注意

次の 3 点に注意する。**(1) 角度の範囲**。`np.arctan2` の値は $-180°$〜$180°$ なので、本編の出力のように 268.3° と表示するには `np.degrees(...) % 360` で 0〜360 の範囲に直す必要がある。**(2) 端点の歪み**。位相が最も知りたい「現在」はデータの末尾にあたるが、フィルタとヒルベルト変換は端点で大きく歪む。周期 48 か月の正確な余弦波で試すと、帯域 42〜54 か月・鏡像パディング 1 周期では末尾の位相が 40〜70° ずれ、帯域 36〜60 か月・パディング 2 周期では数度に収まった（付録 B.5）。帯域が狭いほどフィルタの過渡応答が長くなるためである。**(3) 検算**。本番のデータに使う前に、周期が分かっている合成データで位相が正しく出るかを必ず確かめ、末尾の位相には不確かさの幅を添えて報告する。

参照　補助文書 02 第 4 章。参考記事 [14]（蔵本モデルの位相の推定にヒルベルト変換を使用）。

3.3　命題 1.1 の検証 ― 第 2 回試行

出典：本編第 2 章

背景　第 2 章。位相角と技術世代の遷移データを加えて訓練し直した結果。

コード

```
# ===== 命題1.1検証結果 ―― 光稜精機 第2回試行 =====
# シリコンサイクル位相角 + 技術世代遷移データを追加
mse = 0.0058
print(f"条件1：MSE = {mse:.4f} < 0.01 → 合格 ✓")
knn = 0.87
print(f"条件2：KNN保存率 = {knn:.2f} >= 0.90 → 不合格 ×（大幅改善だが未達）")
min_sv = 0.0024
print(f"条件3：最小特異値 = {min_sv:.4f} >= 0.001 → 合格 ✓")
条件1：MSE = 0.0058 < 0.01 → 合格 ✓
条件2：KNN保存率 = 0.87 >= 0.90 → 不合格 ×（大幅改善だが未達）
条件3：最小特異値 = 0.0024 >= 0.001 → 合格 ✓
```

解説　周期構造を円として与えたことで、近傍関係が 0.63 から 0.87 まで改善し、ヤコビ行列の退化も解消した。残る問題は、技術世代（14 nm と 3 nm など）のデータが別の島に分かれていることだった。

実装上の注意

改善の要因を切り分けるため、位相角だけを加えた場合と、技術世代のデータだけを加えた場合の結果も記録しておくとよい（アブレーション）。

3.4　レシピから物理量への変換

出典：本編第 2 章

背景　第 2 章。伊藤翔太が、装置の運転条件（レシピ）ではなく、レシピが生み出すプラズマの物理量でデータを記述することを提案する。

コード

```
# ===== 技術世代間の連続性 ―― 物理量ベースのデータ変換 =====
# 伊藤翔太の提案
def convert_recipe_to_physics(recipe_params, equipment_model):
    """
    装置のレシピパラメータ（人間が設定する値）を
    プラズマの物理量（自然法則が決める値）に変換する。
    ◆ なぜこれが必要か：
      14nmのレシピと3nmのレシピは、数値としてまったく異なる。
      しかしそれらが生み出すプラズマの物理状態は、
      連続的なパラメータ空間の中で位置が違うだけ。
      レシピ空間では「断絶」に見えるものが、
      物理量空間では「連続」として現れる。
    """
    # プラズマシミュレーション（簡略化モデル）
    electron_density = compute_electron_density(recipe_params, equipment_model)
    ion_energy = compute_ion_energy(recipe_params, equipment_model)
    etch_rate = compute_etch_rate(recipe_params, equipment_model)
    uniformity = compute_uniformity(recipe_params, equipment_model)
    return np.array([electron_density, ion_energy, etch_rate, uniformity])
```

解説　レシピは技術世代ごとに不連続に変わる人間の設定値だが、それが生み出す電子密度・イオンエネルギー・エッチングレート・均一性は物理法則で結ばれた連続な量である。座標を物理量に取り替えることで、世代をまたいで多様体が一つにつながる（補助文書 02 第 5 章）。

実装上の注意

`compute_electron_density` などは本編では中身が省略されている。実装の選択肢は、(1) プラズマの全体モデル（グローバルモデル）による計算、(2) 装置ごとの実験データから作

った応答曲面（代理モデル）、(3) 発光分光分析などで実測した値を直接使う方法、である。(1)(2) を使う場合は、装置の型式ごとに実測値で較正し、モデルの誤差を記録すること。変換に使ったモデルの版も、データと一緒に管理する。

参照　補助文書 04 第 2.3 節。

3.5　命題 1.1 の検証 ― 第 3 回試行（合格）

出典：本編第 2 章

背景　第 2 章。物理量への変換を加えて、3 条件すべてに合格した。

コード

```
# ===== 命題1.1検証結果 ―― 光稜精機 第3回試行 =====
mse = 0.0049
print(f"条件1：MSE = {mse:.4f} < 0.01 → 合格 ✓")
knn = 0.931
print(f"条件2：KNN保存率 = {knn:.3f} >= 0.90 → 合格 ✓")
min_sv = 0.0038
print(f"条件3：最小特異値 = {min_sv:.4f} >= 0.001 → 合格 ✓")
print("\n★ 3条件すべて合格 ★")
print("光稜精機 経営環境多様体は、幾何学的解析に適した")
print("有効な14次元微分可能多様体である。")
```

```
条件1：MSE = 0.0049 < 0.01 → 合格 ✓
条件2：KNN保存率 = 0.931 >= 0.90 → 合格 ✓
条件3：最小特異値 = 0.0038 >= 0.001 → 合格 ✓

★ 3条件すべて合格 ★
光稜精機 経営環境多様体は、幾何学的解析に適した
有効な14次元微分可能多様体である。
```

解説　KNN 保存率 0.931、最小特異値 0.0038 となり、光稜の経営環境多様体は幾何学的解析に使える 14 次元の滑らかな多様体であると判定された。

実装上の注意

合格は「この条件の範囲で使える」ことを示すもので、未来のデータに対する保証ではない。新しいデータが加わるたびに 3 条件を再評価し、悪化の兆候を監視すること。

第4章　ステップ 3〜4：引き戻し計量とスカラー曲率

4.1　引き戻し計量と条件数

出典：本編第 3 章

背景　第 3 章「プルバック計量 ― 半導体の距離感」。多様体の各点で距離の測り方を求め、条件数で異方性を調べる。

コード

```
# ===== プルバック計量の計算 =====
sample_points = vae.encode(X_test)
metrics = [compute_pullback_metric(vae.decoder, z) for z in sample_points]# 条件数の分析
for i, g in enumerate(metrics):
    eigenvalues = np.linalg.eigvalsh(g)
    cn = eigenvalues[-1] / max(eigenvalues[0], 1e-10)
    if cn > 500:
        print(f"警告：点{i}（{dates[i]}）条件数={cn:.0f}")
警告：点287（2019-06-15）条件数=2413 ―― シリコンサイクル不況底
警告：点398（2022-02-28）条件数=1876 ―― ウクライナ戦争＋ネオンガス危機
警告：点512（2025-10-06）条件数=1341 ―― ★現在★ BIS EAR第4弾
警告：点489（2025-03-15）条件数=728 ―― サイバー探索開始
計量計算完了：600サンプル点
平均条件数：89.4
最大条件数：2413（2019年不況底）
```

解説　サンプル点ごとにデコーダのヤコビ行列 J から $g = J^\top J$ を求め、固有値の最大と最小の比（条件数）が 500 を超える点を警告として出力している。2019 年の不況底（2413）と現在（1341）が抽出された。

実装上の注意

`np.linalg.eigvalsh` は固有値を小さい順に返すので、`eigenvalues[-1] / eigenvalues[0]` が条件数になる。$g = J^\top J$ の条件数は J の条件数の 2 乗であることにも注意する。ヤコビ行列は数値差分ではなく `torch.autograd.functional.jacobian` で厳密に求め、曲率で必要になる 2 階微分も自動微分でつなぐのがよい（付録 B.1）。掲載コードの 2 行目末尾では、コメントがコードと同じ行に続いているが、実行には影響しない。

参照　コンパニオン Vol.1 第 2 章（接空間とリーマン計量）、第 11 章。補助文書 02 第 2 章。

4.2　スカラー曲率マップ

出典：本編第 3 章

背景　第 3 章「スカラー曲率 — 危険地帯の地図」。12 年間のサンプル点でスカラー曲率を計算し、不安定な時期を並べる。

コード

```
# スカラー曲率マップ
curvature_map = {}
for i, z in enumerate(sample_points):
    R = compute_scalar_curvature(vae.decoder, z)
    curvature_map[dates[i]] = R
dangerous = [(d, R) for d, R in curvature_map.items() if R < -5.0]
dangerous.sort(key=lambda x: x[1])
print("光稜精機 多様体上の不安定領域 TOP 10：")
for date, R in dangerous[:10]:
    print(f"  {date}：スカラー曲率 R = {R:.2f}")
光稜精機 多様体上の不安定領域 TOP 10：
  2019-06-15：スカラー曲率 R = -62.18  （シリコンサイクル不況底）
  2022-03-01：スカラー曲率 R = -48.73  （ウクライナ戦争＋ネオンガス危機）
  2020-03-20：スカラー曲率 R = -41.25  （COVID第一波）
  2025-10-06：スカラー曲率 R = -33.47  （★現在★ BIS EAR第4弾）
  2023-10-15：スカラー曲率 R = -27.81  （米中対立再激化＋Huawei制裁強化）
  2018-12-01：スカラー曲率 R = -22.39  （米中貿易戦争第1弾）
  2025-03-15：スカラー曲率 R = -18.64  （サイバー探索開始 ★前兆★）
  2020-08-10：スカラー曲率 R = -15.92  （COVID第二波）
  2024-05-20：スカラー曲率 R = -12.47  （台湾総統選後の緊張）
  2021-09-01：スカラー曲率 R = -8.34   （世界的半導体不足ピーク）
```

解説　スカラー曲率が -5 より小さい点を不安定領域として抽出し、値の小さい順に上位 10 件を表示している。不況底（-62.18）、ウクライナ戦争（-48.73）、COVID 第一波（-41.25）に続いて、現在（-33.47）が 4 番目に入った。

実装上の注意

`compute_scalar_curvature` は本編では省略されている。クリストッフェル記号を正しく組み立てる必要があり（付録 B.1）、計量の 2 階微分を含むため自動微分を使う。閾値 -5 は多様体ごとに意味が変わるので、過去の分布（分位点）から決めるか、正規化した値で判断すること。また曲率の点推定だけでなく、MC ドロップアウトで信頼区間を付け、「確信を持って不安定」な点と「判定できない」点を区別する（参考記事 [9]）。どの指標が不安定さに寄与しているかは SHAP で分解できる（参考記事 [10]）。

参照　コンパニオン Vol.1 第 3 章（接続・共変微分・曲率テンソル）、第 11 章、Vol.2 第 1・2 章。参考記事 [9][10]。

第5章　AI エージェントと身体化エージェント

5.1　14 体の AI エージェント・ロスター

出典：本編第 4 章

背景　第 4 章「14 体の AI エージェント」。装置の物理を知覚する身体化エージェント 4 体、経営環境を知覚する非身体化エージェント 9 体、全体を統合するメタエージェント 1 体の構成。

コード

```
光稜精機 14体AIエージェント一覧
═══════════════════════════════════════════════════════════════

【身体化エージェント（4体）—— 装置の物理を知覚する】
Agent_01_Plasma_Guardian
  対象：エッチング装置のプラズマ状態
  センサー：電子密度プローブ、イオンエネルギー分析器、
            発光分光器、RF電力モニター
  World Model：プラズマ物理法則ベース（因果モデル）
  機能：プラズマの異常検知、エッチング品質予測、
        物理法則からの逸脱の検出（サイバー攻撃の物理的検知）

Agent_02_Wafer_Inspector
  対象：ウェハーの物理的品質
  センサー：膜厚計測、線幅計測、欠陥検査（電子顕微鏡）
  World Model：半導体プロセス物理モデル
  機能：歩留まり異常の早期検知、装置間のばらつき検出

Agent_03_Cleanroom_Sentinel
  対象：クリーンルーム環境
  センサー：パーティクルカウンター、温湿度、気圧差、
            振動計（搬送系の異常検知）
  World Model：流体力学＋粒子挙動モデル
  機能：クリーンルーム汚染の早期検知

Agent_04_Supply_Chain_Physical
  対象：部品・資材の物理的品質
  センサー：受入検査装置、X線蛍光分析、材料試験機
  World Model：材料科学モデル
  機能：偽造部品の検知、材料品質の経時変化予測

【非身体化エージェント（9体）—— 経営環境を知覚する】
Agent_05_Market_Sentinel —— 半導体市場動向（SOX指数、設備投資計画）
Agent_06_Regulatory_Radar —— 規制変更（BIS、経産省、EU Chips Act）
Agent_07_Geopolitical_Oracle —— 地政学（台湾海峡、米中関係）
Agent_08_Cyber_Guardian —— サイバー脅威（IT/OTネットワーク監視）
Agent_09_Customer_Pulse —— 主要顧客5社の動向
```

```
Agent_10_Competitor_Watch ―― 競合4社の動向
Agent_11_Financial_Controller ―― 財務・資金管理
Agent_12_Talent_Compass ―― 人材（エンジニア確保・離職リスク）
Agent_13_Innovation_Scout ―― 技術動向（次世代プロセス、新材料）
【メタエージェント（1体）】
Agent_14_Orchestrator ―― 13体の出力を統合、リー括弧による創発効果の検知━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━」
↪  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
```

解説　身体化エージェントは、プラズマ・ウェハー・クリーンルーム・部品の物理的品質を、物理法則に基づく世界モデル（因果モデル）で監視する。非身体化エージェントは、市場・規制・地政学・サイバー・顧客・競合・財務・人材・技術動向を担当する。Agent_14 は 13 体の出力を統合し、リー括弧による創発効果を検知する。

実装上の注意

エージェントの担当を決める際は、時間の尺度、不確実性の構造、意思決定の粒度の 3 つの軸で整理すると、役割の重複や空白を避けやすい（参考記事 [13]）。複数のエージェントが同じ多様体の上で動くと、個々の設計にはない創発的なふるまいが生じうるため、エージェント間の相互作用を監視する仕組みが要る（参考記事 [14]、コンパニオン Vol.2 第 8〜10 章）。

参照　コンパニオン Vol.2 第 6〜10 章。参考記事 [12][13][14]。

5.2　身体化エージェントのアラート（Agent_01）

出典：本編第 5 章

背景　第 5 章「Agent_01 vs Agent_08」。SILICON GHOST が起動した午前 2 時 17 分、身体化エージェントがエッチング装置の改ざんを検知する。

コード

```
# Agent_01_Plasma_Guardian（身体化エージェント）アラート
alert_embodied = {
    'agent': 'Agent_01_Plasma_Guardian',
    'type': 'embodied',
    'timestamp': '2026-09-08T02:17:00+09:00',
    'severity': 'AMBER → RED（3分以内にエスカレーション）',
    'sensor_data': {
        'RF_bias_power': '設定値 300W → 実測値 306.9W（+2.3%）',
        'CF4_flow': '設定値 50 sccm → 実測値 49.15 sccm（-1.7%）',
        'chamber_pressure': '設定値 2.0 Pa → 実測値 2.3 Pa（+0.3 Pa）',
    },
    'world_model_diagnosis': {
        'step_1': (
            '各パラメータの偏差は個別には仕様の許容範囲内。'
```

```
            'RF電力±5%、ガス流量±3%、圧力±0.5Pa。'
            '通常のアラームは発報しない。'
        ),
        'step_2': (
            'しかし、物理法則ベースの因果モデルは以下を予測する：'
            'RF電力+2.3%かつCF4流量-1.7%かつ圧力+0.3Paの'
            '同時偏差は、プラズマ中のイオンエネルギー分布関数（IEDF）を'
            '高エネルギー側にシフトさせ、エッチングレートを'
            '設計値から3.2%増加させる。'
            'この増加は、3nm プロセスのエッチング深さの'
            '仕様公差（±1.5%）を超える。'
        ),
        'step_3': (
            '重要：3つのパラメータが「同時に」「同じ方向に」ずれる'
            '確率は、独立なランダム故障を仮定した場合、'
            '0.023% × 0.017% × 0.03% ≈ 10^{-10}（事実上ゼロ）。'
            '→ これはランダム故障ではない。'
            '→ 制御パラメータの意図的な改ざんの可能性：99.99%以上'
        ),
    },
    'conclusion':
    ↪  'エッチング装置の制御パラメータが外部から改ざんされている可能性が極めて高い。即時停止を推奨。',
}
```

解説　3 つのパラメータ（RF 電力、CF_4 流量、チャンバー圧力）の偏差は、それぞれ単独では許容範囲内だった。しかし物理モデルによれば、3 つの同時の偏差はイオンエネルギー分布を高エネルギー側にずらし、エッチングレートを仕様の公差を超えて増やす。さらに、3 つが同時に同じ方向にずれる確率は、独立なランダム故障では事実上ゼロであるとして、意図的な改ざんの可能性を指摘した。

実装上の注意

本編の確率の計算（各偏差の百分率を掛け合わせる式）は、物語のための簡略な表現である。実運用では、正常運転時のデータから各パラメータの同時分布（相関を含む）を推定し、マハラノビス距離や、物理モデルに基づく尤度で「この組み合わせがどれだけ起こりにくいか」を評価する。また「改ざんの可能性 99.99% 以上」は、事前確率を明示したベイズ的な計算でなければ確率として解釈できない。アラートの根拠（使ったモデル、閾値、データ）を記録し、装置の停止は人間の承認を経て行う設計にする（第 12 章の Agda の例）。

参照　補助文書 02 第 6 章、補助文書 04 第 10 章。参考記事 [12]。

5.3　非身体化エージェントの評価（Agent_08）

出典：本編第 5 章

背景　第 5 章。同じ時刻、ネットワークを監視する非身体化エージェントは「正常」と判断した。

コード

```
# Agent_08_Cyber_Guardian（非身体化エージェント）—— アラートなし
status_disembodied = {
    'agent': 'Agent_08_Cyber_Guardian',
    'type': 'disembodied',
    'timestamp': '2026-09-08T02:17:00+09:00',
    'severity': 'GREEN（正常）',
    'network_monitoring': {
        'IT_network': '異常なし',
        'OT_network': 'トラフィックパターン正常',
        'SIEM_alerts': '0件',
    },
    'assessment': (
        'ネットワーク上に不審なトラフィックなし。'
        'Modbus/TCPコマンドシーケンスは通常の運用パターンと一致。'
        'サイバー脅威レベル：LOW'
    ),
}
```

解説　SILICON GHOST は保守作業の USB メモリから仕込まれ、ネットワークを通じた指令を使わなかったため、通信のパターンにも SIEM のアラートにも痕跡が残らなかった。

実装上の注意

ネットワークの監視は、ネットワークを経由する攻撃しか見えない。USB などの物理的な経路には、装置の持ち込み・保守時のマルウェア検査（SEMI E188）や、物理法則に基づく監視を組み合わせる必要がある。二つのエージェントの判断が食い違ったこと自体を、重要な信号として扱う設計にするとよい（参考記事 [12]）。

参照　補助文書 04 第 9 章。

5.4　知覚空間の位相

出典：本編第 5 章

背景　第 5 章「倉田教授の分析」。二つのエージェントの違いを、知覚空間の位相の違いとして説明する。

コード

```
身体化エージェントと非身体化エージェントの知覚空間の位相
──────────────────────────────────────────────────────────
```

```
【非身体化エージェントの知覚空間 T_d】
  次元：ネットワークトラフィックのパターン空間
  位相：トラフィックパターンの類似度で定義される距離位相
  制限：USBマルウェアの物理的挿入は、この空間には写像されない
        → SILICON GHOSTは T_d の「不可視領域」に存在する

  位相的解釈：
    T_d には「穴」がある ―― USBポート経由の物理的攻撃という
    脅威のクラスが、知覚空間の位相構造から完全に欠落している。
    これは T_d の第一ホモトピー群 π_1(T_d) が自明ではないことを
    意味する ―― 知覚空間を一周しても、元に戻れない経路がある。
    つまり「見えない脅威」が位相的に存在する。

【身体化エージェントの知覚空間 T_e】
  次元：プラズマの物理量の空間
  位相：物理法則が定義する因果的距離位相
  特徴：物理的な異常は、その原因が何であれ、
        物理量の変化として T_e に必ず写像される
        → SILICON GHOSTの効果は T_e では可視

  位相的解釈：
    T_e は物理法則によって「閉じている」――
    物理的な因果の鎖は途切れない。
    装置のパラメータが改ざんされれば、それは必ず
    物理量の変化として現れる。
    T_e の第一ホモトピー群 π_1(T_e) は T_d より「豊か」――
    より多くの脅威クラスを識別できる。
【結論】
  T_d ⊂ T_e（非身体化の知覚空間は、身体化の知覚空間の
  真部分集合である）―― ただしこの包含は集合論的ではなく
  位相的な意味で。
身体化エージェントは、非身体化エージェントが
  「位相的に見えない」脅威を、物理法則を通じて検知できる。
  これが「身体を持つ」ことの位相幾何学的な意味だ。
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
```

解説　非身体化エージェントの知覚空間には、USB 経由の物理的攻撃という「見えない領域」がぽっかり抜けており、身体化エージェントの知覚空間では物理法則がその部分を埋めている、という比喩である。

実装上の注意

本編の説明には、数学的に正確でない表現がある。第一に、穴のある空間で起きるのは「一周しても元に戻れない」ことではなく、「穴をまたいで輪を一点に縮められない」ことである（輪は元の点に戻る）。第二に、穴を持つのは非身体化エージェントの知覚空間なので、穴のない身体化エージェントの知覚空間の基本群は、むしろ単純（自明）になる。「$\pi_1(T_e)$ は T_d より豊か」は、「T_e には T_d にある穴がない」と読み替えるのが正確である。補助文書 02 第 6.2 節で述べたとおり、知覚空間を厳密に位相空間として定義する

ことは研究課題であり、この部分は比喩として読むのが適切である。

参照　補助文書 02 第 6.2 節。

第6章　パーシステント・ホモロジー

6.1　9 つの危機のパーシステント・ホモロジー

出典：本編第 12 章

背景　第 12 章「すべてが 1 つの形を結ぶ」。光稜を襲った 9 つの危機を多様体の上の点として、その「形」を調べる。

コード

```
光稜精機を襲った一連の危機
① BIS EAR第4弾（米国規制）—— 2025年10月
② サイバー探索（APT偵察）—— 2025年〜継続
③ SILICON GHOST仕込み（USBマルウェア）—— 2026年6月
④ SILICON GHOST起動（装置パラメータ改ざん）—— 2026年9月
⑤ 韓国K-半導体装置育成法 —— 2026年10月
⑥ SMIC間接取引スキャンダル —— 2026年11月
⑦ メリディアンEMP攻撃 —— 2027年1月
⑧ リームへの脅迫＋ディープフェイク —— 2027年2〜3月
⑨ リン誘拐未遂 —— 2027年4月
# ===== パーシステント・ホモロジー =====
from ripser import ripser
crisis_points = np.array([z1, z2, z3, z4, z5, z6, z7, z8, z9])
result = ripser(crisis_points, maxdim=2)
print("パーシステント・ホモロジー結果：")
print(f"\nH0（連結成分）：最終的に1つの連結成分に統合")
print(f"  2つのクラスターが距離0.74で接続")
print(f"  クラスターA（技術・サイバー系）：②③④⑦")
print(f"  クラスターB（地政学・規制系）：①⑤⑥")
print(f"  ⑧⑨はクラスターAに近接（メリディアン攻撃系）")
print(f"\nH1（ループ）：2個")
print(f"  ループ1（持続性0.91）：②→③→④→⑦→②")
print(f"    技術攻撃の循環チェーン")
print(f"  ループ2（持続性0.78）：①→⑤→⑥→①")
print(f"    規制・政策の包囲ループ")
パーシステント・ホモロジー結果：

H0：最終的に1つの連結成分。2クラスター構造。
  クラスターA（技術・サイバー系）：②③④⑦⑧⑨
  クラスターB（地政学・規制系）：①⑤⑥

H1：2個のループ

  ループ1（持続性0.91）：サイバー探索→SILICON GHOST→装置攻撃→EMP→サイバー探索
    → 技術攻撃の自己強化ループ。成功するとエスカレートする。

  ループ2（持続性0.78）：BIS規制→韓国国産化→SMIC問題→BIS規制
```

```
    → 規制の包囲ループ。一つの規制が次の規制を誘発する。

そして2つのループの接続点：
  BIS規制（①）が両方のループに属する ――
  米国の規制強化が、技術攻撃のループにも規制包囲のループにも
  「火種」を提供している。
```

解説　9 点に ripser を適用し、H_0（連結成分）で技術・サイバー系と地政学・規制系の 2 つのクラスターを、H_1（ループ）で持続性 0.91 の技術攻撃のループと持続性 0.78 の規制包囲のループを見つけた。BIS 規制が両方のループに属していることが、第 14 章の手術の標的を決めた。

実装上の注意

掲載コードは潜在座標をそのまま ripser に渡しているため、点の間の距離はユークリッド距離になる。GI 理論の考え方に従えば、多様体の上の測地線距離の行列を作り、`ripser(D, distance_matrix=True)` として渡すべきである（付録 B.3）。また 9 点という少ない点では、1 点の位置の誤差でループが現れたり消えたりしうる。点の位置に小さな摂動を加えたり、ブートストラップで点を選び直したりして、ループの持続性が安定しているかを確かめること。2 次元のホモロジー（`maxdim=2`）は、この点数では意味のある結果がほとんど出ないので、計算量を考えれば `maxdim=1` で十分である。

参照　補助文書 02 第 8 章。コンパニオン Vol.2 第 11 章（モース理論と分岐理論）。

第7章　多人数ゲーム理論

7.1　4 者のナッシュ均衡

出典：本編第 13 章

背景　第 13 章「3 つの国旗」。米国・中国・ロシア・光稜の 4 者のゲームとして状況を整理し、ナッシュ均衡を求める。

コード

```
nash_equilibria = compute_nash_equilibria(
    players=['US', 'China', 'Russia', 'Koryo'],
    strategies=strategy_sets,
    payoff_functions=manifold_derived_payoffs,
)
print("ナッシュ均衡分析（最も蓋然性の高い均衡）：")
print("  US：規制維持＋同盟国支援（弧月型の構造転換）")
print("  China：技術内製化の加速＋間接的妨害の継続")
print("  Russia：サイバー攻撃の低強度維持（コスト増大により縮小）")
print("  Koryo：ポートフォリオ手術＋日米蘭三角同盟＋自社セキュリティ強化")
print()
```

```
print("  光稜の8年後企業価値：1兆2,400億円（現在比+51.2%）")
print("  反事実（現状維持）：4,800億円（現在比-41.5%）")
```

解説　各プレイヤーの戦略の組ごとの利得を、経営環境多様体の上のベクトル場の作用から推定し（manifold_derived_payoffs）、最も蓋然性の高い均衡を求めている。光稜の戦略（手術・同盟・防御）が、他のプレイヤーの最善の応答を変えることが要点である。

実装上の注意

compute_nash_equilibria は本編では省略されている。4 人以上の有限ゲームのナッシュ均衡は、Gambit（Python からは pygambit）の解法で計算できる（nashpy は 2 人ゲーム向け）。均衡が複数ある場合は、どれを「最も蓋然性が高い」とするか（リスク支配、過去の行動との整合など）の基準を明示する。各国の利得は推定値なので、利得を少しずつ変えて均衡が変わらないかを確かめる感度分析が欠かせない。なお、本編第 13 章の 8 年後の企業価値（1 兆 2,400 億円）と第 15 章の値（1 兆 4,100 億円）は、前者が他のプレイヤーの応答を含む均衡の評価、後者が手術後の軌道のシミュレーションという、異なる前提による推定である。

参照　補助文書 02 第 9 章。コンパニオン Vol.2 第 23 章（情報の対称性と非対称性）。

第8章　手術理論によるポートフォリオの再編

8.1　手術計画

出典：本編第 14 章

背景　第 14 章「手術理論」。SMIC 向け事業などを切り取り、日米蘭の新しい顧客を貼り合わせる計画。

コード

```
surgery_plan = {
    'cut': [
        'SMIC向け全事業（直接・間接とも完全撤退）',
        '韓国低収益保守契約3件（国産化圧力で採算割れ）',
    ],
    'glue': [
        'Rapidus（日本）向けEUV周辺装置 —— 日本半導体復興の中核',
        'Intel新工場（オハイオ、アリゾナ）向けエッチング装置 —— CHIPS法補助金',
        'ASML（オランダ）との技術提携深化 —— EUV次世代装置の共同開発',
        'TSMC熊本工場向け —— 台湾リスクの分散',
    ],
    'cobordism_check': '切除境界と接着境界の技術仕様が±5%で一致。充足。',
}
print("手術後のポートフォリオ：")
print("  台湾TSMC：25% → 18%（台湾本島15%＋熊本3%）")
```

```
print("  韓国：23% → 16%（Samsung集中を解消）")
print("  米国Intel：12% → 20%（CHIPS法活用）")
print("  中国SMIC：7% → 0%（完全撤退）")
print("  日本Rapidus：3% → 12%（半導体復興）")
print("  オランダASML連携：5% → 10%")
print("  その他：25% → 24%")
```

解説　切除（cut）と接着（glue）の対象を列挙し、手術後の顧客構成を示している。顧客構成は手術の前後とも合計 100% になっている（前：25+23+12+7+3+5+25、後：18+16+20+0+12+10+24）。

実装上の注意

cobordism_check は、数学のコボルディズムの概念を「切り取る事業で浮いた人・設備・技術が、貼り合わせる事業でそのまま使えるか」という実務の確認に置き換えた比喩的な用法であり、数学の定理がこの計画を保証するわけではない。数学的に検証できるのは、手術後のデータで多様体を作り直し、パーシステント・ホモロジーを計算し直したときに、規制包囲のループが本当に消えるかどうかである。手術理論は、AI エージェントが使う道具ではなく、位相の変化を理解するための記述言語として位置づけるのが適切である（参考記事 [15]）。

参照　補助文書 02 第 10 章。コンパニオン Vol.2 第 13 章。参考記事 [5][6][7][15]。

第9章　シンプレクティック積分

9.1　8 年間の航海図

出典：本編第 15 章

背景　第 15 章「シンプレクティック航海図」。手術後の光稜の 8 年間（96 か月）の軌道を、シュテルマー＝ヴェルレ法で計算する。

コード

```
# シュテルマー＝ヴェルレ法による8年間のシミュレーション
trajectory = simulate_symplectic(
    z_initial=z_post_surgery,
    time_horizon=96,  # 96ヶ月 = 8年
    dt=1.0,
    method='stormer_verlet',
)
H_initial = compute_hamiltonian(z_post_surgery, p_initial)
H_final = compute_hamiltonian(trajectory[-1], p_final)
print(f"ハミルトニアン保存性：ΔH/H = {abs(H_final-H_initial)/H_initial:.2e}")
ハミルトニアン保存性：ΔH/H = 3.14×10^-8
【手術あり】
  企業価値（現在）：8,200億円
```

```
  企業価値（4年後）：1兆400億円（+26.8%）
  企業価値（8年後）：1兆4,100億円（+72.0%）
  営業利益率：18.2% → 24.5%
【手術なし】
  企業価値（4年後）：5,900億円（-28.0%）
  企業価値（8年後）：3,200億円（-61.0%）
差分（8年後）：1兆900億円
手術コスト：320億円
ROI：3,306%
```

解説　ハミルトニアンの相対誤差 $\Delta H/H = 3.14 \times 10^{-8}$ は、96 か月の計算で数値的なずれがほとんど生じていないことを示す。手術ありとなしの 8 年後の企業価値の差は 1 兆 900 億円（1 兆 4,100 億円 − 3,200 億円）で、ROI は $(10{,}900 - 320)/320 \approx 33.06$、すなわち 3,306% である（差分から手術コストを引いた純益を、手術コストで割った値）。

実装上の注意

陽的なシュテルマー＝ヴェルレ法がシンプレクティックになるのは、ハミルトニアンが位置の部分と運動量の部分に分かれる場合である。リーマン多様体の上の運動のように運動エネルギーが位置に依存する場合は、陰的な一般化リープフロッグ法を使う（付録 B.4）。相対誤差の分母は、ハミルトニアンが負になりうるため `abs(H_initial)` とすること。また $\Delta H/H$ が小さいことは計算の正しさを示すだけで、モデルの正しさは示さない。8 年先の予測には、MC ドロップアウトなどで不確実性の幅を付けて報告する（参考記事 [9]）。

参照　補助文書 02 第 11 章。コンパニオン Vol.2 第 15 章、Vol.1 第 7 章（Neural ODE）。参考記事 [7][9]。

第10章　層理論による内部の不整合の検知

10.1　組織の層ラプラシアン

出典：本編第 16 章

背景　第 16 章「層理論による検知」。組織構造のグラフの上に層を作り、記録の継ぎ目の食い違いを社員ごとのスコアとして求める。

コード

```
# 層理論による整合性チェック
sheaf = construct_org_sheaf(koryo_employee_data, access_logs)
L = sheaf.laplacian()
for employee in all_employees:
    x = sheaf.get_data_vector(employee)
    inconsistency = np.linalg.norm(L @ x)
    if inconsistency > threshold:
```

```
        print(f"不整合検知：{employee.name}，スコア={inconsistency:.2f}")
不整合検知：宮下達也（常務），スコア=8.74
不整合検知：高橋修一（中国事業部次長），スコア=6.21
不整合検知：周明（上海オフィスマネージャー），スコア=7.89
```

解説　各社員（頂点）に決裁記録・アクセス範囲・受注と出荷・通関書類などのデータを置き、上司と部下、部署と部署の関係（辺）ごとに「両端の記録は一致すべき」という規則（制限写像）を定めて、層ラプラシアン L を作る。食い違いが集まる社員ほどスコアが大きくなる。

実装上の注意

掲載コードでは、社員一人分のデータベクトル `x` に組織全体の層ラプラシアン `L` を掛けているが、これでは行列とベクトルの大きさが合わない。正しくは、組織全体のデータを一つのベクトルに並べて L を一度だけ作用させ、その結果のうち各社員に対応するブロックの大きさをスコアとする（付録 B.6）。付録 B.6 では、補助文書 02 第 12.4 節の 3 部署の演習問題を実際に計算し、$(L_{\mathcal{F}}x)_A = (0, -30)$、$(L_{\mathcal{F}}x)_B = (0, -30)$、$(L_{\mathcal{F}}x)_C = (0, 60)$ を確かめている。閾値は正常期間のスコアの分布から決める。スコアは「調べるべき場所」を示すだけで不正の認定ではないため、本編のとおり、コンプライアンス委員会の承認、目的と範囲の限定、社外の弁護士の立ち会い、当事者の弁明の機会といった手続きと組み合わせて運用すること。

参照　補助文書 02 第 12 章、補助文書 04 第 7 章。コンパニオン Vol.2 第 18 章、Vol.1 第 8 章。参考記事 [1][2][3][4]。

第11章　座標系の変換（本編の呼称：ゲージ変換）

11.1　秘密の回転による流出データの無効化

出典：本編第 17 章

背景　第 17 章「三方面の反撃」。サイドチャネル攻撃で盗まれた可能性のある、古い座標で書かれた情報を無効にするため、潜在空間を秘密の回転で新しい座標系に移す。

コード

```
# ゲージ変換 —— 座標系の回転
R = random_orthogonal_matrix(dim=14, seed=secure_random_seed)
vae_new = apply_gauge_transformation(vae, R)
# 多様体の幾何学的性質は不変
curvature_old = compute_scalar_curvature(vae.decoder, z_current)
curvature_new = compute_scalar_curvature(vae_new.decoder, R @ z_current)
print(f"旧座標系の曲率：{curvature_old:.4f}")
print(f"新座標系の曲率：{curvature_new:.4f}")
print(f"差分：{abs(curvature_old - curvature_new):.6f}")
旧座標系の曲率：-8.3421
```

```
新座標系の曲率：-8.3422
差分：0.000011（数値誤差の範囲内）
→ 幾何学的性質は完全に保存。
  しかし潜在空間の座標値は完全に変更。
  盗まれたデータは無効化された。
```

解説　14 次元のランダムな直交行列 R で潜在空間を回転し、デコーダを新しい座標系で再パラメータ化する。変換の前後でスカラー曲率が一致すること（-8.3421 と -8.3422）を確かめ、幾何学的な性質が保たれたまま座標値だけが変わったことを示している。

実装上の注意

新しいデコーダは $f_{\text{new}}(w) = f(R^{\top} w)$（$w = Rz$）と定義する必要がある。このとき引き戻し計量は $g_{\text{new}} = R\, g\, R^{\top}$ となり、固有値や曲率などの幾何学的な量は厳密に保たれる（付録 B.7 で数値的に確認）。平行移動 b を加えて $w = Rz + b$ としてもよい。秘密の回転は鍵と同じであり、安全な端末の中で生成し、厳格に保管し、定期的に更新する。回転は線形変換なので、古い座標と新しい座標の対応が数組でも漏れると R が推定されうる。また、生データそのものが流出していれば、多様体をゼロから作り直されてしまうため、この方法は生データの保護や物理的な遮蔽と組み合わせて使う。

参照　補助文書 02 第 13 章。コンパニオン Vol.2 第 16 章（用語の背景）。

第12章　形式検証と ZKML

12.1　形式検証の結果

出典：本編第 18 章

背景　第 18 章「最終証明」。14 体のエージェントについて、3 つの道具で 3 つの異なることを証明する。

コード

```
形式検証結果：
  Coq証明：安全性上界定理 ―― 検証済み ✓
  Z3 SMT：制約充足可能 ―― 検証済み ✓
  Agda依存型：エージェント挙動の型安全性 ―― 検証済み ✓
```

解説　Coq で安全性上界定理（どんな入力でも推奨が上限を超えないこと、Lipschitz 定数が 0.8 以下であること）、Z3 で 14 体の制約が同時に満たせること（充足可能性）、Agda の依存型で人間の承認の証明なしに装置停止の命令を組み立てられないことを示した。

実装上の注意

ニューラルネットワークの Lipschitz 定数は、各層の重みのスペクトルノルムの積で上から抑えられる（活性化関数が tanh などで Lipschitz 定数 1 の場合）。付録 B.8 に、スペクトル正規化で上界を 0.8 以下にする例と、Z3 で制約の充足可能性を判定し、制約を追加すると充足不能（unsat）になる例を示す。Coq での証明では浮動小数点の誤差を扱う必要があり（Flocq など）、PyTorch のモデルは ONNX を経由して証明系の表現に変換するのが現実的な手順である（参考記事 [16]）。形式検証が保証するのは「書いた仕様どおりに動く」ことであり、仕様の誤りや、実行環境へのマルウェアの侵入は防げない。セキュリティ対策と併用すること。

参照　補助文書 02 第 14 章。コンパニオン Vol.2 第 24〜29 章。参考記事 [16]。

12.2　ZKML 証明の生成

出典：本編第 18 章

背景　第 18 章。分析が約束したモデルとデータから正しく計算されたことを、データの中身を明かさずに証明する。

コード

```
print("ZKML証明生成：")
print("  回路サイズ：4.2M ゲート（光稜の多様体は14次元で最大規模）")
print("  証明生成時間：1時間52分")
print("  証明サイズ：2.8 MB")
print("  検証時間：1.4秒")
print("  結果：証明有効 ✓")
```

解説　回路サイズ 4.2M ゲートの証明を 1 時間 52 分で生成し、検証は 1.4 秒で済んだ。証明を作るのは重く、確かめるのは一瞬というゼロ知識証明の非対称性を表している。

実装上の注意

掲載コードは結果を表示しているだけである。実装には EZKL などのツールを使い、モデルを ONNX 形式で書き出して回路に変換する。計算は有限体の上で行われるため、浮動小数点の値を固定小数点に量子化する必要があり、量子化による精度の変化を必ず評価する。現在の技術では、パイプライン全体を証明すると計算量が大きくなりすぎるため、VAE のデコーダ、因果効果の最終的な推定、不確実性の計算など、重要な部分に絞って証明する。

参照　補助文書 02 第 14.6 節。コンパニオン Vol.2 第 4・5 章。

さらに学ぶために　著者（Étale Cohomology）は、Qiita で、定理証明言語（Coq/Rocq、Lean 4、Agda）、SMT ソルバー、関数型言語、圏論と論理に関する記事を多数公開している。形式検証の考え方や、証明支援系の使い方を実際に手を動かして学びたい読者は、著者の Qiita のページ（https://qiita.com/etale_cohomology）を参照されたい。

付録 A　推奨ライブラリ一覧

分野	ライブラリ	本書での用途
数値計算	NumPy, SciPy, pandas	行列計算、固有値、信号処理（バンドパス、ヒルベルト変換）
深層学習	PyTorch	VAE、自動微分によるヤコビ行列・曲率
Neural ODE	torchdiffeq	連続時間のダイナミクス
機械学習	scikit-learn	近傍探索、固有次元推定
因果推論	EconML	DML による因果効果の推定
説明可能性	SHAP	要因分解
位相的データ解析	ripser, persim	パーシステント・ホモロジー
ゲーム理論	Gambit (pygambit), nashpy	ナッシュ均衡（nashpy は 2 人ゲーム向け）
層理論	自作実装（付録 B.6）、または PySheaf 等	層ラプラシアン
形式検証	Coq/Rocq, Lean 4, Agda, Z3 (z3-solver)	安全性制約の証明
ZKML	EZKL など	証明の生成と検証

インストールの例：

```
pip install numpy scipy pandas scikit-learn torch torchdiffeq
pip install econml shap ripser persim nashpy pygambit z3-solver
```

付録 B　補足の参照実装

本付録のコードは、本文の「実装上の注意」で述べた点を示すための最小限の例であり、本編の数値を再現するものではない。B.5〜B.8 は実際に実行し、出力を確認したものである。

B.1　自動微分によるクリストッフェル記号

```
import torch

def metric(decoder, z):
    J = torch.autograd.functional.jacobian(decoder, z, create_graph=True)  # (D, d)
    return J.T @ J                                                          # (d, d)

def christoffel(decoder, z):
    z = z.detach().requires_grad_(True)
    g = metric(decoder, z)
    g_inv = torch.linalg.inv(g)
    d = z.shape[0]
    # dg[l, i, j] = ∂_l g_ij（すべての方向 l について計算する）
    dg = torch.stack([torch.autograd.grad(g[i, j], z, create_graph=True)[0]
                      for i in range(d) for j in range(d)], dim=1).reshape(d, d, d)
    A = dg                    # A[i, j, l] = ∂_i g_jl
    B = dg.permute(1, 0, 2)   # B[i, j, l] = ∂_j g_il
    C = dg.permute(1, 2, 0)   # C[i, j, l] = ∂_l g_ij
    # Γ^k_ij = 1/2 g^{kl} (∂_i g_jl + ∂_j g_il − ∂_l g_ij)
    return 0.5 * torch.einsum('kl,ijl->kij', g_inv, A + B - C)
```

実装後は、平面の極座標や球面など、クリストッフェル記号が解析的に分かっている例で必ず検算すること。

B.2　状態依存のベクトル場と自動微分によるリー括弧

```
import torch

def X_equipment(z):
    # 装置異常の影響：中国向け依存度（次元 0）が高い状態ほど打撃が大きくなる例
    w = torch.sigmoid(3.0 * z[0])
    c = torch.zeros_like(z); c[0], c[3] = -0.6, 0.4   # 押す方向（定数）
    return w * c                                       # 押す強さは状態 z に依存

def lie_bracket(X, Y, z):
    # [X, Y]^k = X^i ∂_i Y^k − Y^i ∂_i X^k
    JX = torch.autograd.functional.jacobian(X, z, create_graph=True)  # JX[k, i] = ∂_i X^k
    JY = torch.autograd.functional.jacobian(Y, z, create_graph=True)
    return JY @ X(z) - JX @ Y(z)
```

ベクトル場が定数なら、リー括弧は恒等的に 0 になる。創発効果を測るには、状態に依存す

るベクトル場を使う必要がある。

B.3　測地線距離行列によるパーシステント・ホモロジー

```
import numpy as np
from ripser import ripser

n = len(points)                        # 9 つの危機の潜在座標
D = np.zeros((n, n))
for a in range(n):
    for b in range(a + 1, n):
        D[a, b] = D[b, a] = geodesic_distance(points[a], points[b])  # 測地線距離

result = ripser(D, distance_matrix=True, maxdim=1)
# 安定性の確認：点の位置に小さな摂動を加えて H1 の本数と持続性が変わらないかを見る
```

B.4　位置依存のハミルトニアンに対する一般化リープフロッグ法

運動エネルギーが $\frac{1}{2} g^{ij}(z) p_i p_j$ のように位置に依存する場合、次の陰的な 3 段の更新を用いるとシンプレクティック性が保たれる（Leimkuhler & Reich, 2004）。

$$
\begin{aligned}
p_{n+1/2} &= p_n - \tfrac{\Delta t}{2}\, \partial_z H(z_n, p_{n+1/2}) && \text{（陰的：反復で解く）}\\
z_{n+1} &= z_n + \tfrac{\Delta t}{2}\left[\partial_p H(z_n, p_{n+1/2}) + \partial_p H(z_{n+1}, p_{n+1/2})\right] && \text{（陰的：反復で解く）}\\
p_{n+1} &= p_{n+1/2} - \tfrac{\Delta t}{2}\, \partial_z H(z_{n+1}, p_{n+1/2}) && \text{（陽的）}
\end{aligned}
$$

長期シミュレーションでは、相対誤差 $|H_n - H_0|/|H_0|$ を時系列で記録し、ドリフトがないことを確認する。

B.5　ヒルベルト変換によるシリコンサイクルの位相（端点の扱いを含む）

```
import numpy as np
from scipy.signal import butter, filtfilt, hilbert

def silicon_cycle_phase(order_data, period_months=(36, 60), pad_cycles=2):
    """受注データ（月次）からシリコンサイクルの瞬時位相（度，0〜360）を推定する。"""
    x = np.asarray(order_data, float)
    x = x - np.polyval(np.polyfit(np.arange(len(x)), x, 1), np.arange(len(x)))  # 線形トレンド除去
    fs = 1.0                                   # 月次データ：1 サンプル／月
    low, high = 1.0 / period_months[1], 1.0 / period_months[0]   # 周波数（1/月）
    b, a = butter(3, [low / (fs / 2), high / (fs / 2)], btype='band')
    pad = int(pad_cycles * period_months[1])   # 端点の歪みを減らすための鏡像パディング
    xp = np.concatenate([x[pad:0:-1], x, x[-2:-pad-2:-1]])
    c = filtfilt(b, a, xp)[pad:pad + len(x)]
    phase = np.angle(hilbert(c))               # −π〜π
    return np.degrees(phase) % 360.0, np.cos(phase), np.sin(phase)

# 検算：周期 48 か月の余弦波なら、位相は時刻に比例して増える
```

```
t = np.arange(144)
deg, c, s = silicon_cycle_phase(100 + 10 * np.cos(2 * np.pi * t / 48))
print(np.round(deg[48:53], 1), "期待値:", np.round((360 * t[48:53] / 48) % 360, 1))
print("末尾:", np.round(deg[-3:],1), "期待値:", np.round((360 * t[-3:] / 48) % 360, 1))
```

実行結果：

```
[ 0.1  7.6 15.1 22.6 30.1] 期待値: [ 0.   7.5 15.  22.5 30. ]
末尾: [340.2 347.7 356.1] 期待値: [337.5 345.  352.5]
```

周期 48 か月の正確な余弦波では、途中の位相はほぼ一致し、末尾でも誤差は数度に収まる。帯域を 42〜54 か月、パディングを 1 周期にすると、末尾の誤差は 40〜70° に広がった（本文第 3.2 節）。実データでは、末尾の位相に不確かさの幅を添えて報告すること。

B.6　層ラプラシアンと頂点ごとのスコア（3 部署の演習問題）

```
import numpy as np

def sheaf_laplacian(n_nodes, stalk_dim, edges):
    """edges: [(u, v, F_u, F_v)]。F_u, F_v は頂点のデータを辺の上で比べる制限写像（行列）。"""
    rows = []
    for u, v, Fu, Fv in edges:
        r = np.zeros((Fu.shape[0], n_nodes * stalk_dim))
        r[:, u*stalk_dim:(u+1)*stalk_dim] = -Fu
        r[:, v*stalk_dim:(v+1)*stalk_dim] = Fv
        rows.append(r)
    delta = np.vstack(rows)          # 余境界作用素 δ
    return delta, delta.T @ delta    # 層ラプラシアン L = δ^T δ

def node_scores(L, x, n_nodes, stalk_dim):
    """全体のデータ x に L を一度だけ作用させ、頂点ごとのブロックの大きさをスコアにする。"""
    Lx = (L @ x).reshape(n_nodes, stalk_dim)
    return Lx, np.linalg.norm(Lx, axis=1)

# 3 部署の演習問題：A(営業), B(物流), C(経理)。各部署は（数量, 単価）を持つ
I = np.eye(2)
edges = [(0, 1, I, I), (1, 2, I, I), (2, 0, I, I)]   # 両端の記録は一致すべき
x = np.array([10, 50,  10, 50,  10, 80], float)
delta, L = sheaf_laplacian(3, 2, edges)
Lx, score = node_scores(L, x, 3, 2)
print("L x =", Lx.tolist())
print("スコア =", np.round(score, 1).tolist(), " 整合性半径 =", round(float(np.linalg.norm(delta @ x)), 2))
```

実行結果：

```
L x = [[0.0, -30.0], [0.0, -30.0], [0.0, 60.0]]
スコア = [30.0, 30.0, 60.0]  整合性半径 = 42.43
```

不整合は経理部 C の単価の記録に集中している。社員単位のデータでも同じように、組織全体のベクトルに L を一度だけ作用させ、各頂点のブロックの大きさをスコアとする。

B.7　座標系の変換で計量の固有値が保たれることの確認

```
import numpy as np
rng = np.random.default_rng(0)
W1, W2 = rng.normal(size=(32, 14)), rng.normal(size=(130, 32))
def f(z):                       # 例として使う滑らかなデコーダ R^14 → R^130
    return W2 @ np.tanh(W1 @ z)

def jacobian(fun, z, eps=1e-6):
    return np.stack([(fun(z + eps*e) - fun(z - eps*e)) / (2*eps) for e in np.eye(len(z))], axis=1)

# 秘密の回転 Q（QR 分解で一様な直交行列を作り、行列式を +1 にそろえる）
Q, Rr = np.linalg.qr(rng.normal(size=(14, 14)))
Q = Q @ np.diag(np.sign(np.diag(Rr)))
if np.linalg.det(Q) < 0: Q[:, 0] *= -1
b = rng.normal(size=14)

f_new = lambda w: f(Q.T @ (w - b))         # 新しいデコーダ f'(w) = f(Q^T(w − b))
z = rng.normal(size=14); w = Q @ z + b       # 同じ点の新しい座標

g  = jacobian(f, z).T @ jacobian(f, z)
gn = jacobian(f_new, w).T @ jacobian(f_new, w)
print("g' = Q g Q^T :", np.allclose(gn, Q @ g @ Q.T, atol=1e-5))
print("固有値（条件数）は不変:", np.allclose(np.linalg.eigvalsh(g), np.linalg.eigvalsh(gn), atol=1e-5))
print("座標の値は変わる:", np.round(z[:3], 3), "→", np.round(w[:3], 3))
```

実行結果：

```
g' = Q g Q^T : True
固有値（条件数）は不変: True
座標の値は変わる: [ 1.819 -0.126  1.954] → [ 1.827 -0.237  1.7  ]
```

B.8　Lipschitz 定数の上界と Z3 による充足可能性の判定

```
import numpy as np
# (1) Lipschitz 定数の上界：各層のスペクトルノルムの積（活性化関数は tanh などで Lipschitz 定数 1）
rng = np.random.default_rng(1)
Ws = [rng.normal(size=(64, 14)), rng.normal(size=(64, 64)), rng.normal(size=(8, 64))]
Ws = [W / (np.linalg.norm(W, 2) / 0.92) for W in Ws]        # スペクトル正規化（各層 0.92 に）
bound = np.prod([np.linalg.norm(W, 2) for W in Ws])
print(f"Lipschitz 上界 = {bound:.3f}  (≤ 0.8: {bound <= 0.8})")

# (2) Z3：複数の制約が同時に満たせるか（充足可能か）
from z3 import Real, Solver, sat
inv, ship, cash = Real('inventory'), Real('shipment'), Real('cash')
```

```
s = Solver()
s.add(inv >= 120)                # 在庫の下限
s.add(ship <= inv - 100)         # 出荷しても安全在庫 100 を割らない
s.add(ship >= 30)                # 主要顧客への最低出荷
s.add(cash >= 50 - 0.2 * ship)   # 資金繰りの制約
print("充足可能:", s.check() == sat, "  例:", s.model())
s.add(inv <= 125)                # 制約を足して矛盾させてみる（在庫上限）
print("在庫上限を追加後:", s.check())
```

実行結果：

```
Lipschitz 上界 = 0.779  (≤ 0.8: True)
充足可能: True   例: [inventory = 130, shipment = 30, cash = 44]
在庫上限を追加後: unsat
```

上界は各層のスペクトルノルムの積なので、実際の Lipschitz 定数はこれ以下である。Z3 の例では、在庫の下限・安全在庫・最低出荷の制約に在庫の上限 125 を加えると、同時に満たせなくなる（unsat）。14 体のエージェントの制約でも同じように、矛盾がないことを機械的に確かめられる。

付録 C　コード索引

ステップ	本書の節	主なコード片
—	第 2 章	2 層アーキテクチャ、データ基盤
1–2	第 3 章	命題 1.1 の検証（3 回）、シリコンサイクルの位相角、レシピから物理量への変換
3–4	第 4 章	引き戻し計量、条件数、スカラー曲率
エージェント	第 5 章	14 体のロスター、身体化・非身体化のアラート、知覚空間の位相
拡張	第 6〜11 章	パーシステント・ホモロジー、ゲーム理論、手術理論、シンプレクティック積分、層ラプラシアン、座標系の変換
信頼性	第 12 章	形式検証、ZKML
付録	B.1〜B.8	クリストッフェル記号、リー括弧、測地線距離の PH、一般化リープフロッグ法、位相の推定、層ラプラシアン、座標変換、Lipschitz 上界と Z3